

CMP-7037A/6060A Network Security and IoT

Assessment 1: Design and Analyse Secure Networks

Name: John Callaghan
Student ID: 100384938

Assessment 1 – Task Sheet A: Network Setup and Analysis

1. Network Simulation Setup

I began by creating the network topology in Cisco Packet Tracer. This involved adding the required routers, switches, and end devices. To enable the serial connection between the routers, I physically added an HWIC-2T serial interface module into slot 1 of both the R1-ISP and R2-Central 1841 routers.

Following the addressing table, all devices were configured with their static IP addresses, subnet masks, and default gateways to establish the network.

Screenshots:

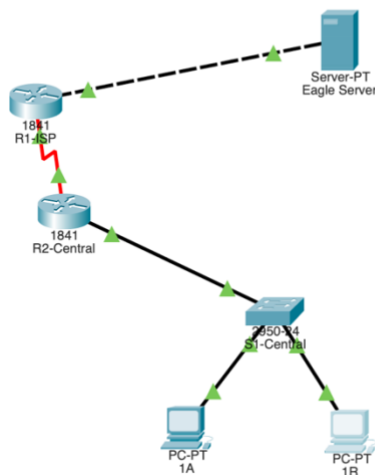


Figure 1: Task A Network Topology

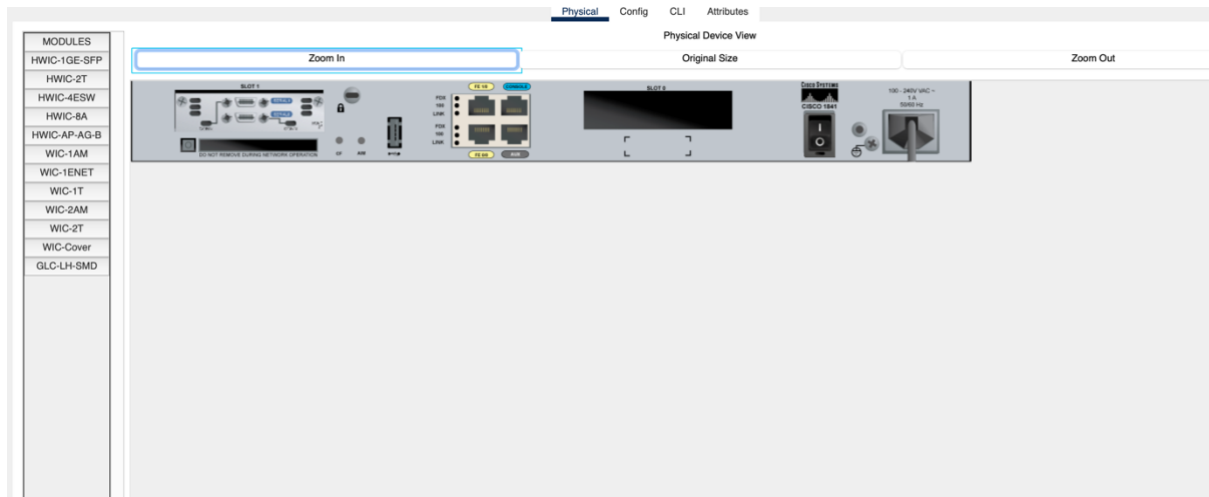


Figure 2: Adding HWIC-2T Module to R1-ISP

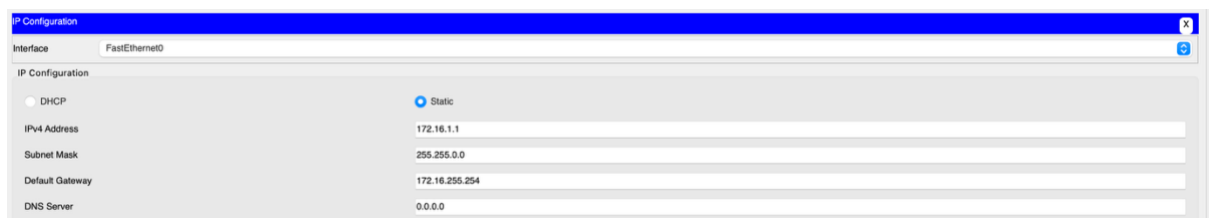


Figure 3: IP Configuration of PC 1A

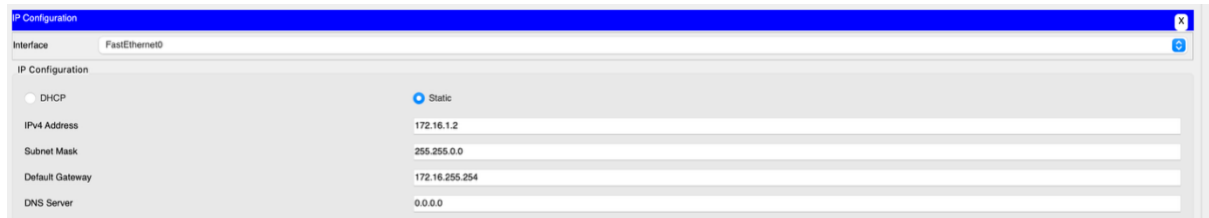


Figure 4: IP Configuration of PC 1R

2. Routing Configuration

To enable end-to-end communication between the two separate networks 172.16.0.0/16 and 192.168.254.0/24, static routing was configured.

- On R1-ISP: A static route was added for the 172.16.0.0/16 network, directing traffic to R2-Central 10.10.10.5 as the next hop.
- On R2-Central: A static route was added for the 192.168.254.0/24 network, directing traffic to R1-ISP 10.10.10.6 as the next hop.

Connectivity was verified with successful ping tests between PC 1R and the Eagle Server.

Screenshots:

```

Router>enable
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#hostname R1-ISP
R1-ISP(config)#int fa0/0
R1-ISP(config-if)#- [ ] ip address 192.168.254.253 255.255.25
^
% Invalid input detected at '^' marker.

R1-ISP(config-if)#ip address 192.168.254.253 255.255.255.0
R1-ISP(config-if)#no shutdown

R1-ISP(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
exit
R1-ISP(config)#int se0/0/0
%Invalid interface type and number
R1-ISP(config)#interface Se0/0/0
%Invalid interface type and number
R1-ISP(config)#int Se0/1/0
R1-ISP(config-if)#ip address 10.10.10.6 255.255.255.252
R1-ISP(config-if)#clock rate 64000
R1-ISP(config-if)#no shutdown

%LINK-5-CHANGED: Interface Serial0/1/0, changed state to down
R1-ISP(config-if)#exit
R1-ISP(config)#exit
R1-ISP#
%SYS-5-CONFIG_I: Configured from console by console
show ip interface brief

```

Interface	IP-Address	OK?	Method	Status	Protocol
FastEthernet0/0	192.168.254.253	YES	manual	up	up
FastEthernet0/1	unassigned	YES	unset	administratively down	down
Serial0/1/0	10.10.10.6	YES	manual	down	down
Serial0/1/1	unassigned	YES	unset	administratively down	down
Vlan1	unassigned	YES	unset	administratively down	down

```

R1-ISP#

```

Figure 5: CLI Interface Configuration for R1-ISP

```

--- System Configuration Dialog ---

Would you like to enter the initial configuration dialog? [yes/no]:

Press RETURN to get started!

Router>enable
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#hostname R2-Central
R2-Central(config)#int se0/1/0
R2-Central(config-if)#ip address 10.10.10.5 255.255.255.252
R2-Central(config-if)#no shutdown

R2-Central(config-if)#
%LINK-5-CHANGED: Interface Serial0/1/0, changed state to up
exit
R2-Central(config)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial0/1/0, changed state to up
int fa0/0
R2-Central(config-if)#ip address 172.16.255.254 255.255.0.0
R2-Central(config-if)#no shutdown

R2-Central(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
exit
R2-Central(config)#exit
R2-Central#
%SYS-5-CONFIG_I: Configured from console by console
show ip interface brief

```

Interface	IP-Address	OK?	Method	Status	Protocol
FastEthernet0/0	172.16.255.254	YES	manual	up	up
FastEthernet0/1	unassigned	YES	unset	administratively down	down
Serial0/1/0	10.10.10.5	YES	manual	up	up
Serial0/1/1	unassigned	YES	unset	administratively down	down
Vlan1	unassigned	YES	unset	administratively down	down

```

R2-Central#

```

Figure 6: CLI Interface Configuration for R2-Central

Routing Table for R1-ISP			Routing Table for R2-Central		
Type	Network	Port	Type	Network	Port
C	10.10.10.4/30	Serial0/1/0	C	10.10.10.4/30	Serial0/1/0
S	172.16.0.0/16	---	C	172.16.0.0/16	FastEthernet0/0
C	192.168.254.0/24	FastEthernet0/0	S	192.168.254.0/24	---

Figure 7: Static Routing Tables for R1-ISP and R2-Central

3. ICMP Packet Analysis (PC 1R to Eagle Server)

To analyse the packet's journey, the simulation was set up to only capture ICMP packets. An ICMP echo request (ping) was sent from PC 1R to the Eagle Server. The simulation event list showed the packet's step-by-step path:

1. **Request Path:** PC 1R , S1-Central , R2-Central , R1-ISP , Eagle Server
2. **Reply Path:** Eagle Server , R1-ISP , R2-Central , S1-Central , PC 1R

This confirmed that the routing was correct and showed the journey for a complete ping.

Screenshots:

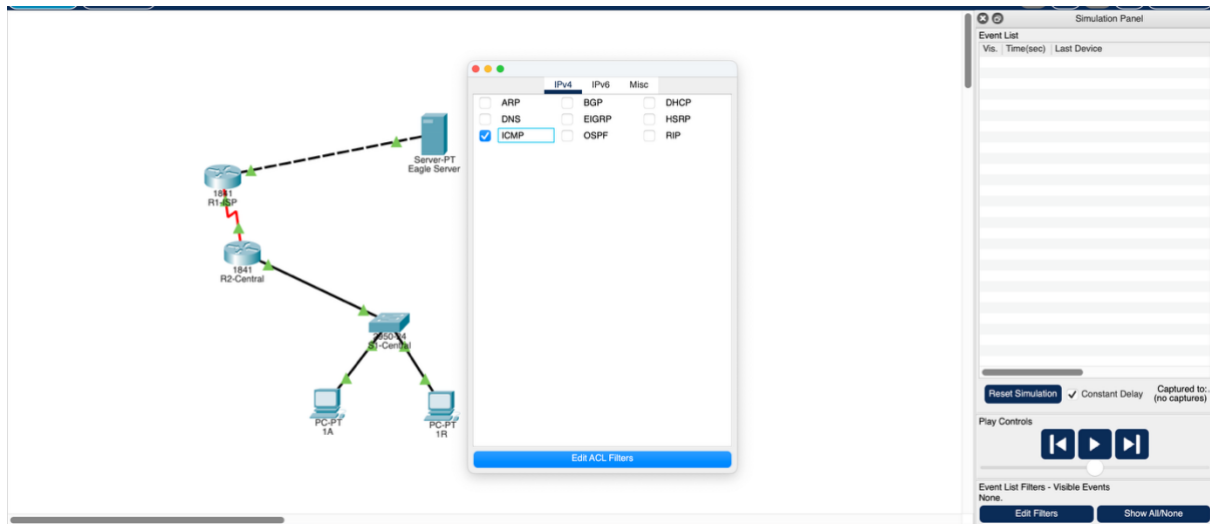


Figure 8: ICMP Filter Enabled in Simulation Panel

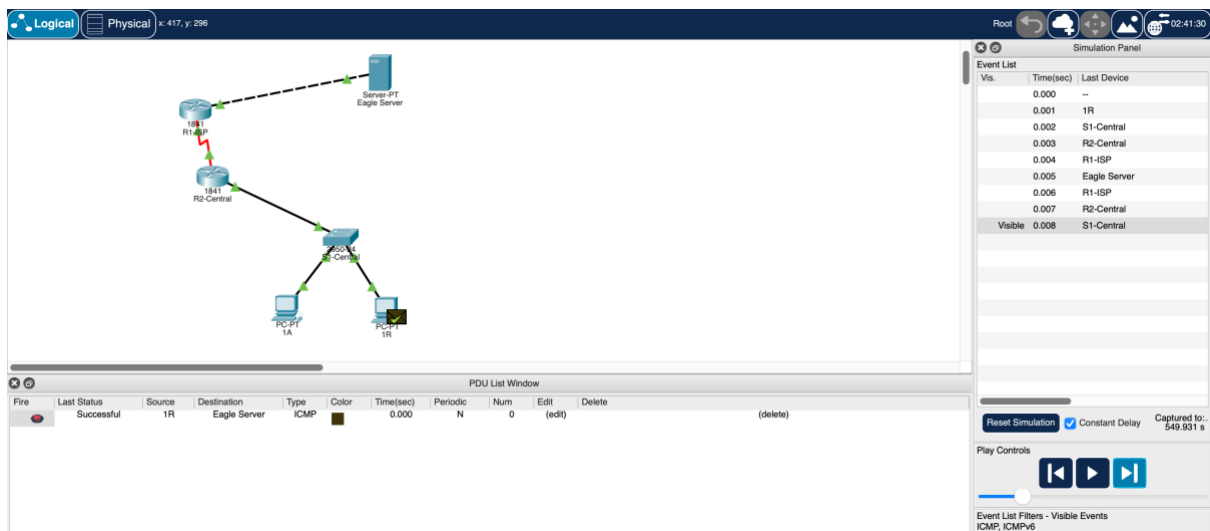


Figure 9: successful ICMP ping in Real time Mode

Simulation Panel		
Event List		
Time(sec)	Last Device	
0.000	--	
0.001	1R	
0.002	S1-Central	
0.003	R2-Central	
0.004	R1-ISP	
0.005	Eagle Server	
0.006	R1-ISP	
0.007	R2-Central	
0.008	S1-Central	

Figure 10: Full event List

PDU Information at Device: R1-ISP

OSI Model

Inbound PDU Details

Outbound PDU Details

At Device: R1-ISP

Source: 1R

Destination: Eagle Server

In Layers

Layer7

Layer6

Layer5

Layer4

Layer 3: IP Header Src. IP: 172.16.1.2, Dest. IP: 192.168.254.254 ICMP Message Type: 8

Layer 2: HDLC Frame HDLC

Layer 1: Port Serial0/1/0

Out Layers

Layer7

Layer6

Layer5

Layer4

Layer 3: IP Header Src. IP: 172.16.1.2, Dest. IP: 192.168.254.254 ICMP Message Type: 8

Layer 2: Ethernet II Header 000A.4160.1801 >> 0060.5C05.9D99

Layer 1: Port(s): FastEthernet0/0

1. The routing table finds a routing entry to the destination IP address.

2. The destination network is directly connected. The device sets destination as the next-hop.

3. The device decrements the TTL on the packet.

Figure 11:PDU Information at R1-ISP

4. Further Packet Analysis

I sent a new PDU from PC 1A to the Eagle Server and used the simulation to inspect the packet at each hop.

- Observation: The Layer 3 IP headers were constant, showing the original source 172.16.1.1 and final destination 192.168.254.254 for the entire journey. In contrast, the Layer 2 MAC headers were rewritten at each router hop. The packet left PC 1A with R2's MAC as the destination, and it left R2 with R1's MAC as the destination.

Screenshots:



Figure 12: ARP Packet from PC 1A

PDU Information at Device: S1-Central

OSI Model

Inbound PDU Details

Outbound PDU Details

At Device: S1-Central

Source: 1A

Destination: Broadcast

In Layers

Layer7

Layer6

Layer5

Layer4

Layer3

Layer 2: Ethernet II Header
0030.F208.4201 >> 000A.
4191.7789 ARP Packet Src. IP:
172.16.255.254, Dest. IP:
172.16.1.1

Layer 1: Port FastEthernet0/1

Out Layers

Layer7

Layer6

Layer5

Layer4

Layer3

Layer 2: Ethernet II Header
0030.F208.4201 >> 000A.
4191.7789 ARP Packet Src. IP:
172.16.255.254, Dest. IP:
172.16.1.1

Layer 1: Port(s): FastEthernet0/2

1. The frame source MAC address was found in the MAC table of Switch.

2. This is a unicast frame. Switch looks in its MAC table for the destination MAC address.

Challenge Me

<< Previous Layer

Next Layer >>

Figure 13: Successful PDU Event List PC-A to Eagle Server

4b. Combination 1: Switch-Router Analysis

I created a Simple PDU from S1-Central to its default gateway, R2-Central, to analyze a switch-router communication path.

Observation: The test was successful. By inspecting the PDU at R2-Central, I could see the inbound and outbound frame details. The inbound packet's Layer 3 destination IP 172.16.255.254 matched the router's interface, so the router processed the packet itself as an ICMP echo request instead of forwarding it.

Screenshots:

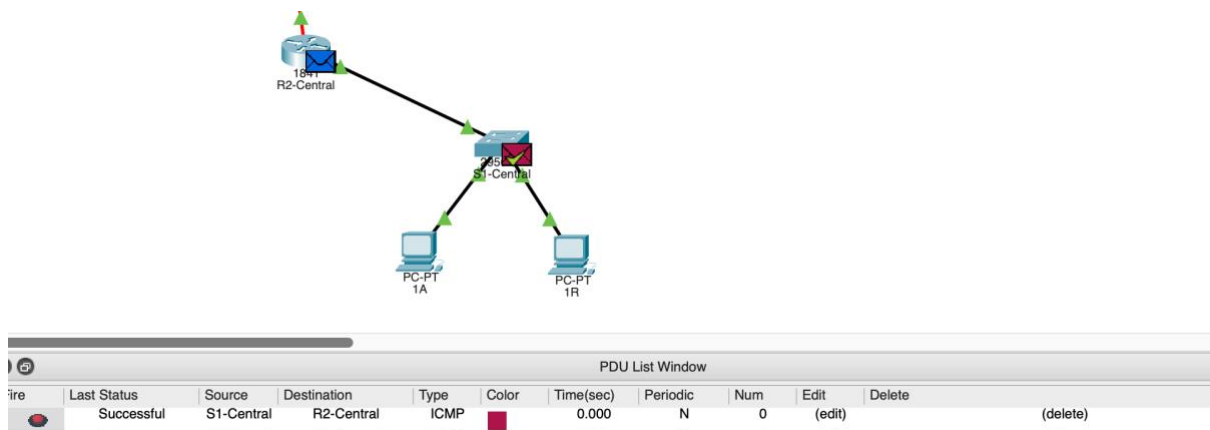


Figure 14: Successful ICMP between S1 and r2

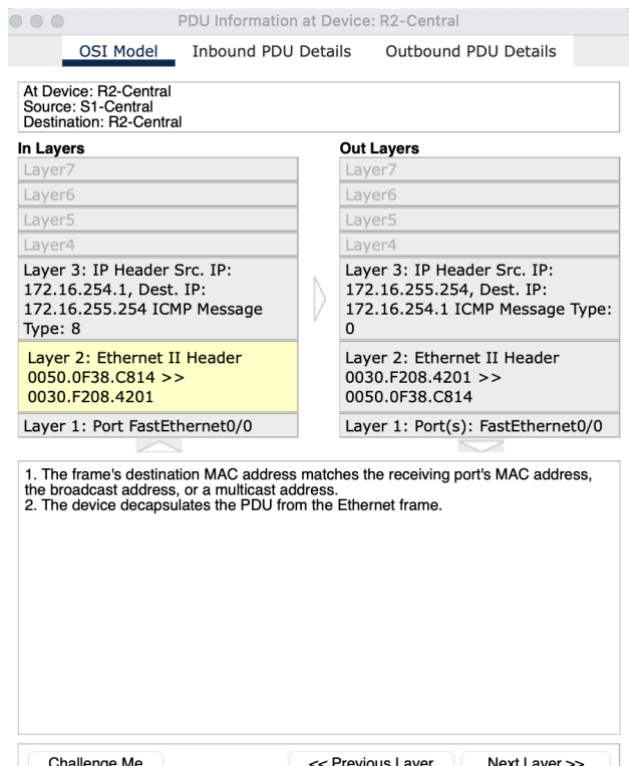


Figure 15:PDU Details at R2-Central

4c. HTTP Request Analysis

I confirmed the HTTP service was enabled on the Eagle Server and then used PC 1R's web browser to access <http://192.168.254.254>.

- Observation: The packet types were completely different from a ping. Instead of ICMP, the event list was immediately populated with **TCP** and **HTTP** packets. This is because HTTP is a Layer 7 protocol that relies on TCP (Layer 4) for a reliable connection. The simulation showed the TCP three-way handshake (SYN, SYN-ACK, ACK) occurring before any HTTP "GET" data was actually sent.

Screenshots:

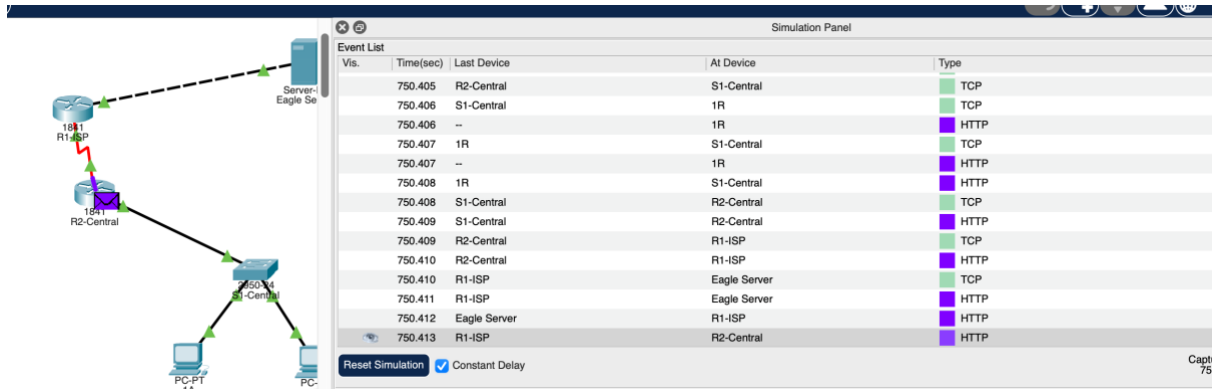


Figure 18: Event List Showing HTTP Packets and TCP

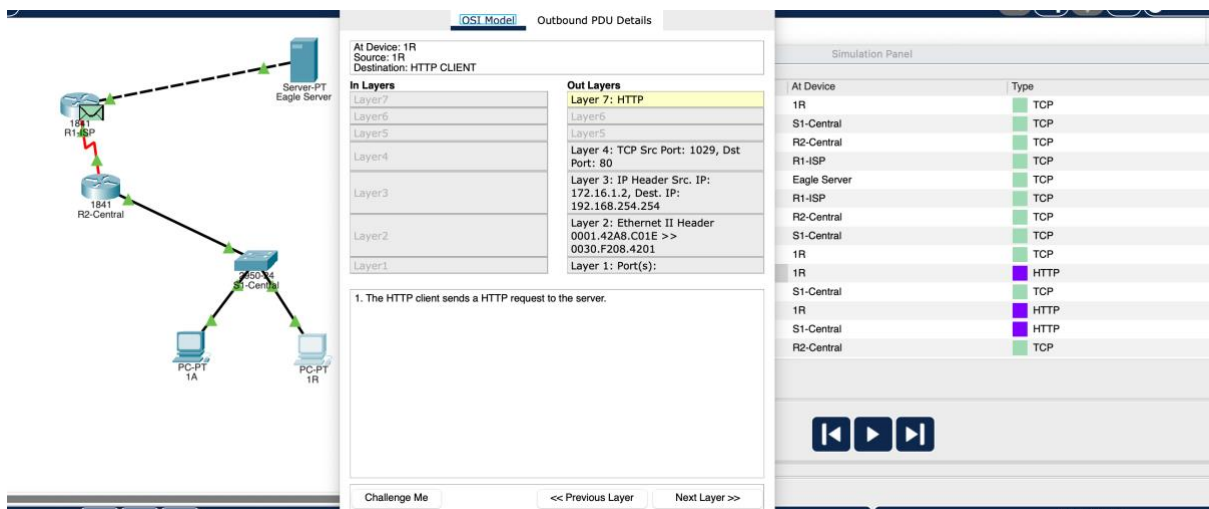


Figure 19:PDU Information Showing Layer 4 (TCP) and Layer 7 (HTTP)

6. Task A Reflection

This task provided a fundamental understanding of how data travels across a network. The main focus was the difference between Layer 2 and Layer 3 addressing; seeing the IP layer 3 headers remain constant, while the MAC layer 2 headers were stripped and rebuilt at each router hop. It also showed the practical difference between a simple protocol like ICMP one request, one reply and a more complex connection protocol like HTTP, which requires a full TCP three-way handshake before any data can be sent.

Assessment 1 – Task Sheet B: Access Control Lists

1. Simulation Overview

For this task, the four-LAN topology was built. Before implementing any ACLs, I configured static routing on R1 and R2 to ensure full connectivity between all four LANs. My specific interface setup, confirmed with show ip interface brief, was:

- **R1 Setup:** Gig0/1 (Yellow LAN), Gig0/2 (Green LAN)
- **R2 Setup:** Gig0/1 (Blue LAN), Gig0/2 (Pink LAN)

Screenshots:

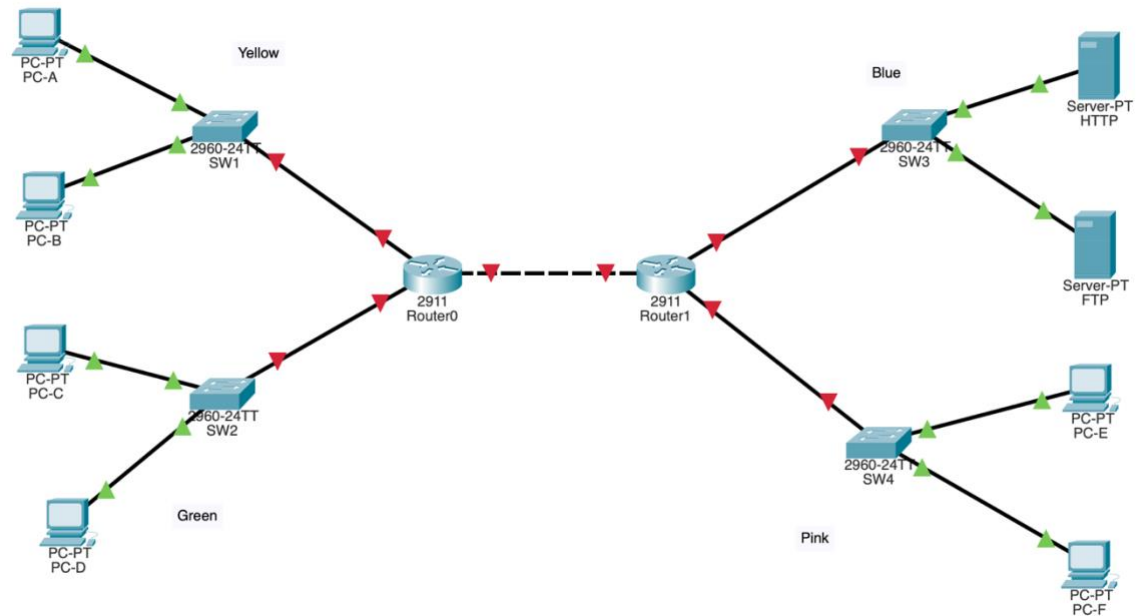


Figure 20: Task sheet B topology

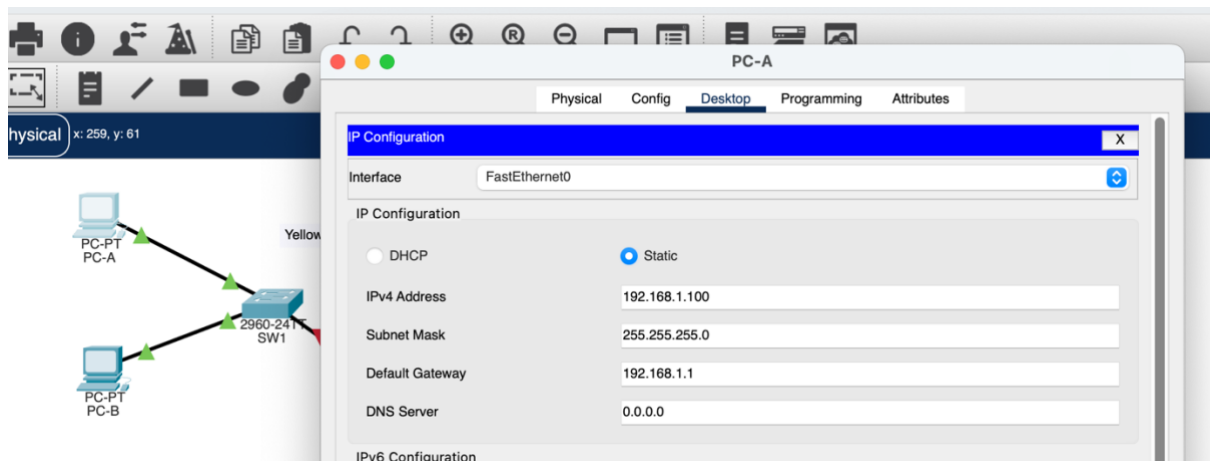
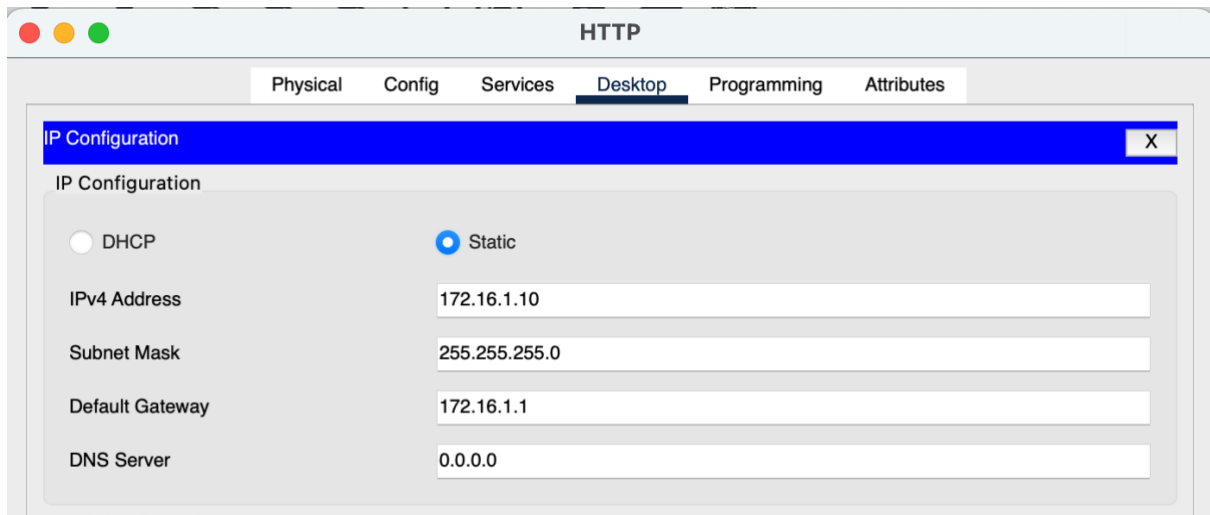


Figure 21: Setting up PC-A IP config



HTTP

Physical Config Services **Desktop** Programming Attributes

IP Configuration [X]

IP Configuration

☐ DHCP
 ☒ Static

IPv4 Address: 172.16.1.10

Subnet Mask: 255.255.255.0

Default Gateway: 172.16.1.1

DNS Server: 0.0.0.0

Figure 22: Setting up HTTP Server

FTP

Service ☒ On ☐ Off

User Setup

Username Password

☐ Write
 ☐ Read
 ☐ Delete
 ☐ Rename
 ☐ List

Username	Password	Permission

Figure 23: Setting up FTP Server

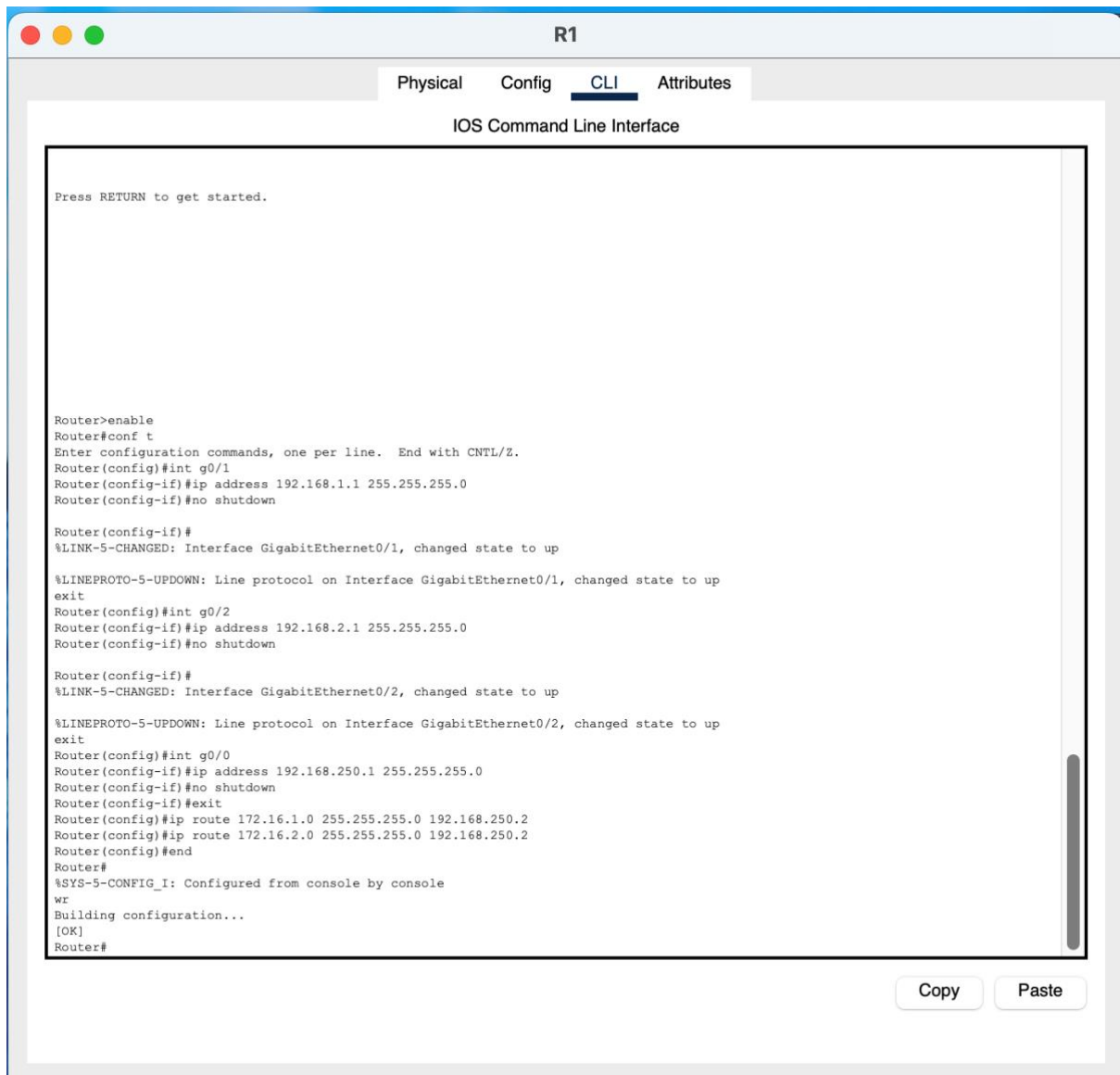


Figure 24: CLI Configuration of R1 Static routes

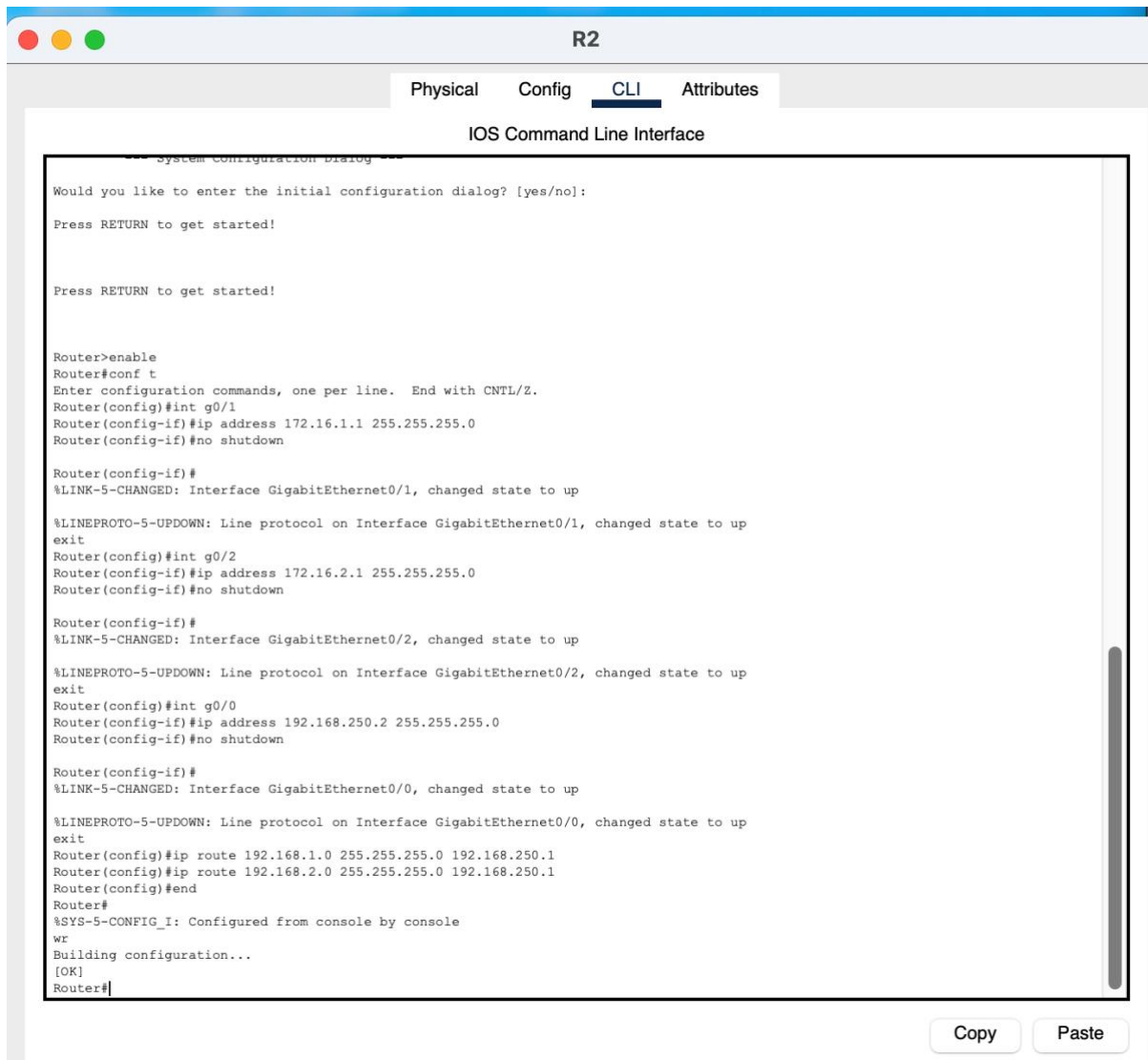


Figure 25: CLI Configuration of R2 Static Routes

```

Router#
Router#Show ip interface brief

```

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0	192.168.250.1	YES	manual	up	up
GigabitEthernet0/1	192.168.1.1	YES	manual	up	up
GigabitEthernet0/2	192.168.2.1	YES	manual	up	up
Vlan1	unassigned	YES	unset	administratively down	down

```

Router#

Router>show ip interface brief

```

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0	192.168.250.2	YES	manual	up	up
GigabitEthernet0/1	172.16.1.1	YES	manual	up	up
GigabitEthernet0/2	172.16.2.1	YES	manual	up	up
Vlan1	unassigned	YES	unset	administratively down	down

```

Router>|

```

Figure 26: Ip Routes R1 + R2

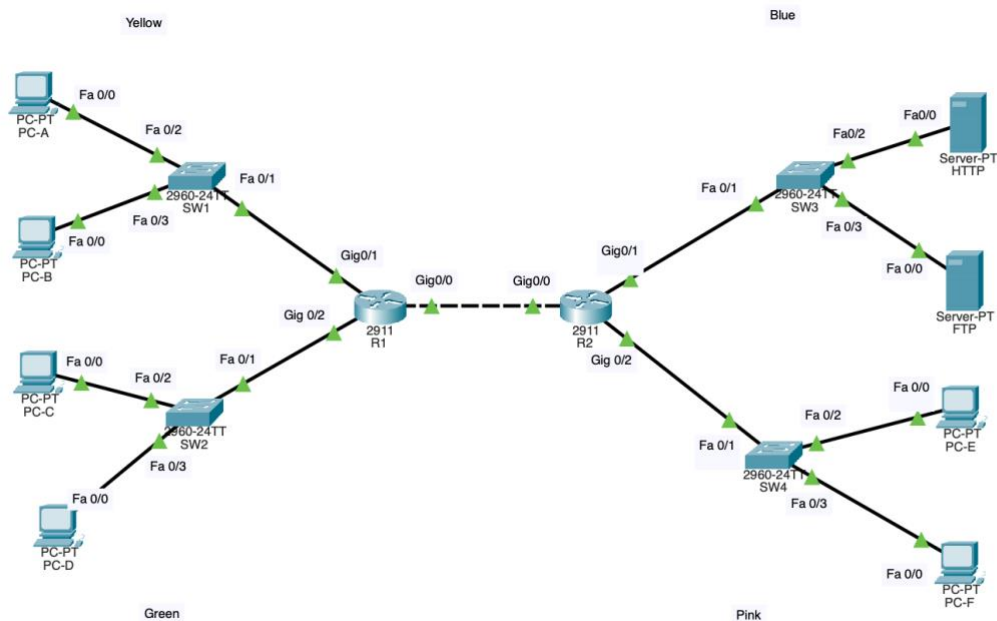


Figure 27: Fully configured Topology

2. Standard IPv4 ACL 101

To restrict access to the Pink LAN, I configured access-list 10. The rules were to permit PC-C, the first half of the Yellow LAN, and the Blue LAN servers.

For a standard ACL, the best practice is to apply it as close to the destination as possible. Therefore, the ACL was applied on R2, to interface GigabitEthernet0/2 my Pink LAN interface, in the out direction.

Screenshots:

```
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no access-list 10
Router(config)#access-list 10 permit host 192.168.2.50
Router(config)#access-list 10 permit 192.168.1.0 0.0.0.127
Router(config)#access-list 10 permit host 172.16.1.10
Router(config)#access-list 10 permit host 172.16.1.20
Router(config)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
show access-lists
Standard IP access list 10
 10 permit host 192.168.2.50
 20 permit 192.168.1.0 0.0.0.127
 30 permit host 172.16.1.10
 40 permit host 172.16.1.20
Router#
```

Figure 28: Configured ACL 10 on R2 CLI

3. Testing Standard ACL 10

I tested the ACL by pinging PC-E in the Pink LAN from various hosts.

- Permitted: Pings from PC-A (first half of Yellow) and PC-C were Successful.
- Denied: Pings from PC-B (second half of Yellow) and PC-D Failed, returning a "Destination host unreachable" message from the router.

These tests confirm all rules of ACL 10 were working correctly.

Screenshots:

```
C:\>ping 172.16.2.100

Pinging 172.16.2.100 with 32 bytes of data:

Reply from 172.16.2.100: bytes=32 time=30ms TTL=126
Reply from 172.16.2.100: bytes=32 time<1ms TTL=126
Reply from 172.16.2.100: bytes=32 time<1ms TTL=126
Reply from 172.16.2.100: bytes=32 time<1ms TTL=126

Ping statistics for 172.16.2.100:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 30ms, Average = 7ms
```

Figure 29: Successful Ping from PC-A

```
C:\>
C:\>ping 172.16.2.100

Pinging 172.16.2.100 with 32 bytes of data:

Reply from 192.168.250.2: Destination host unreachable.
Reply from 192.168.250.2: Destination host unreachable.
Reply from 192.168.250.2: Destination host unreachable.
Reply from 192.168.250.2: Destination host unreachable.

Ping statistics for 172.16.2.100:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>
```

Figure 30: Failed Ping from PC-B

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 172.16.2.100

Pinging 172.16.2.100 with 32 bytes of data:

Reply from 172.16.2.100: bytes=32 time=2ms TTL=126
Reply from 172.16.2.100: bytes=32 time=23ms TTL=126
Reply from 172.16.2.100: bytes=32 time<1ms TTL=126
Reply from 172.16.2.100: bytes=32 time<1ms TTL=126

Ping statistics for 172.16.2.100:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 23ms, Average = 6ms

C:\>
```

Figure 31: Successful Ping from PC-C

```

Cisco Packet Tracer PC Command Line 1.0
C:\>ping 172.16.2.100

Pinging 172.16.2.100 with 32 bytes of data:

Reply from 192.168.250.2: Destination host unreachable.
Reply from 192.168.250.2: Destination host unreachable.
Reply from 192.168.250.2: Destination host unreachable.
Reply from 192.168.250.2: Destination host unreachable.

Ping statistics for 172.16.2.100:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>

```

Figure 32: Failed Ping from PC-D

PDU List Window										
Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC-A	PC-E	ICMP		0.000	N	0	(edit)	(delete)
	Failed	PC-B	PC-E	ICMP		0.000	N	1	(edit)	(delete)
	Successful	PC-C	PC-E	ICMP		0.000	N	2	(edit)	(delete)
	Failed	PC-D	PC-E	ICMP		0.000	N	3	(edit)	(delete)

Figure 33: Event List

4. Extended IPv4 ACL

To restrict access to the Blue LAN, I configured access-list 199. The rule was to deny the Yellow LAN from accessing the HTTP server 172.16.1.10 but permit all other traffic.

For an extended ACL, the best practice is to apply it as close to the source as possible. Therefore, the ACL was applied on R1, to interface GigabitEthernet0/1 which is my Yellow LAN interface, in the in direction.

Screenshots:

```

Router>enable
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#access-list 199 deny tcp 192.168.1.0 0.0.0.255 host 172.16.1.10 eq 80
Router(config)#access-list 199 permit ip any any
Router(config)#int g0/1
Router(config-if)#ip access-group 199 in
Router(config-if)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console

-
Router#show access-lists
Extended IP access list 199
 10 deny tcp 192.168.1.0 0.0.0.255 host 172.16.1.10 eq www
 20 permit ip any any

```

Figure 34: CLI Configuration of ACL 199 on R1

5. Testing Extended ACL 199

I tested this ACL using both the web browser and the command prompt:

- HTTP Test - Denied: PC-A (Yellow LAN) tried to access `http://172.16.1.10` and Failed ("Request Timeout").
- ICMP Test - Permitted: PC-A (Yellow LAN) pinged 172.16.1.10 and was Successful.
- HTTP Test - Permitted: PC-C (Green LAN) tried to access `http://172.16.1.10` and was Successful.

This confirms the ACL correctly filtered *only* TCP port 80 HTTP from *only* the Yellow LAN.

Screenshots:

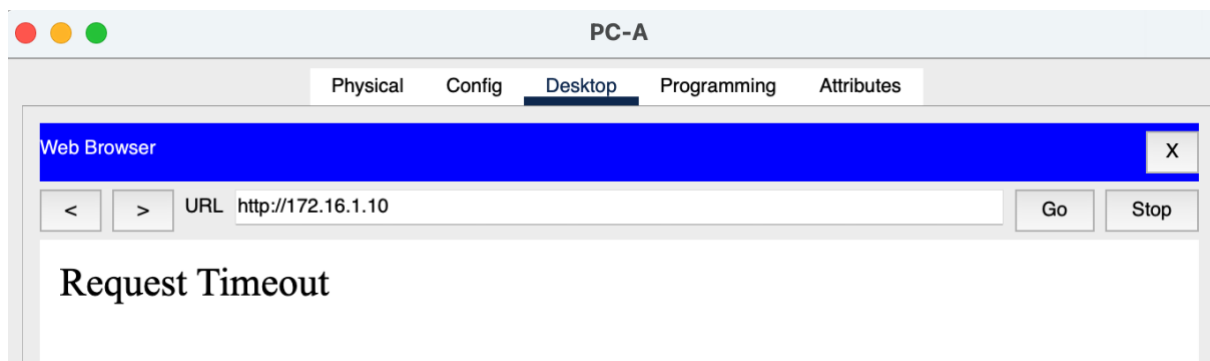


Figure 35: Failed HTTP Test from PC-A

```
C:\>ping 172.16.1.10

Pinging 172.16.1.10 with 32 bytes of data:

Reply from 172.16.1.10: bytes=32 time=1ms TTL=126
Reply from 172.16.1.10: bytes=32 time<1ms TTL=126
Reply from 172.16.1.10: bytes=32 time<1ms TTL=126
Reply from 172.16.1.10: bytes=32 time<1ms TTL=126

Ping statistics for 172.16.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

Figure 36: Successful Ping from PC-A



Figure 37: Successful HTTP Test from PC-C

6. Standard ACL: Deny Pink to Yellow

For the next task I was required to write a new standard ACL to deny the Pink LAN from reaching the Yellow LAN, while allowing all other LANs.

Based on the "close to destination" rule, I applied access-list 11 on R1, outbound on the Yellow LAN interface Gig0/1. The test results confirmed the Pink LAN (PC-E) was blocked, while the Green LAN (PC-C) was still permitted.

Screenshots:

```
Router>enable
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#access-list 11 deny 172.16.2.0 0.0.0.255
Router(config)#access-list 11 permit any
Router(config)#int g0/1
Router(config-if)#ip access-group 11 out
Router(config-if)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
```

Figure 38: CLI Configuration of ACL 11 on R1

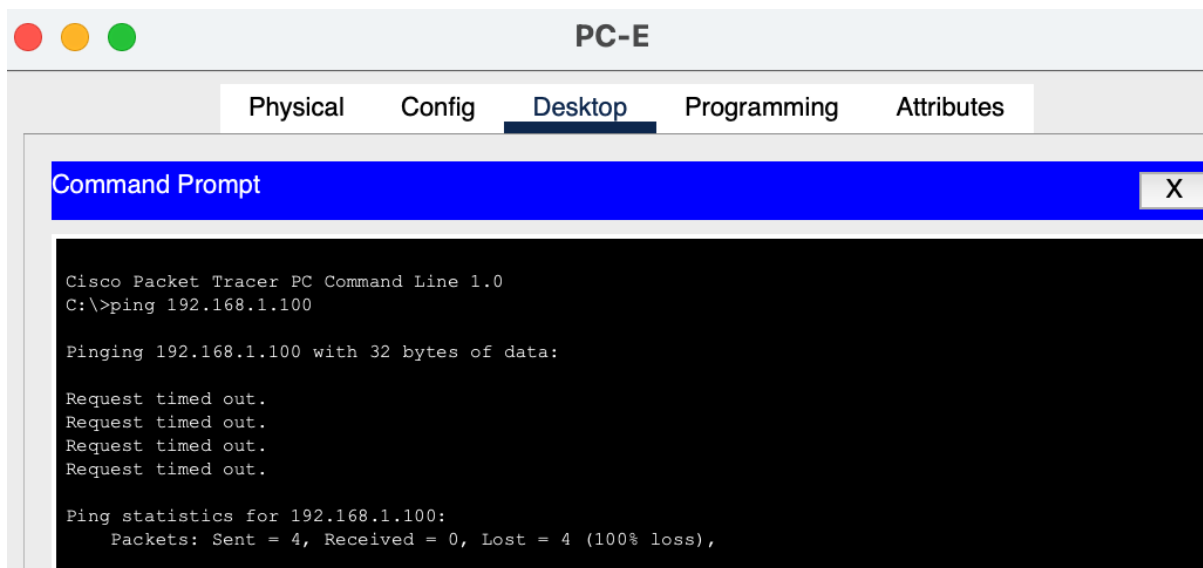


Figure 39: Failed Ping from PC-E /Pink LAN

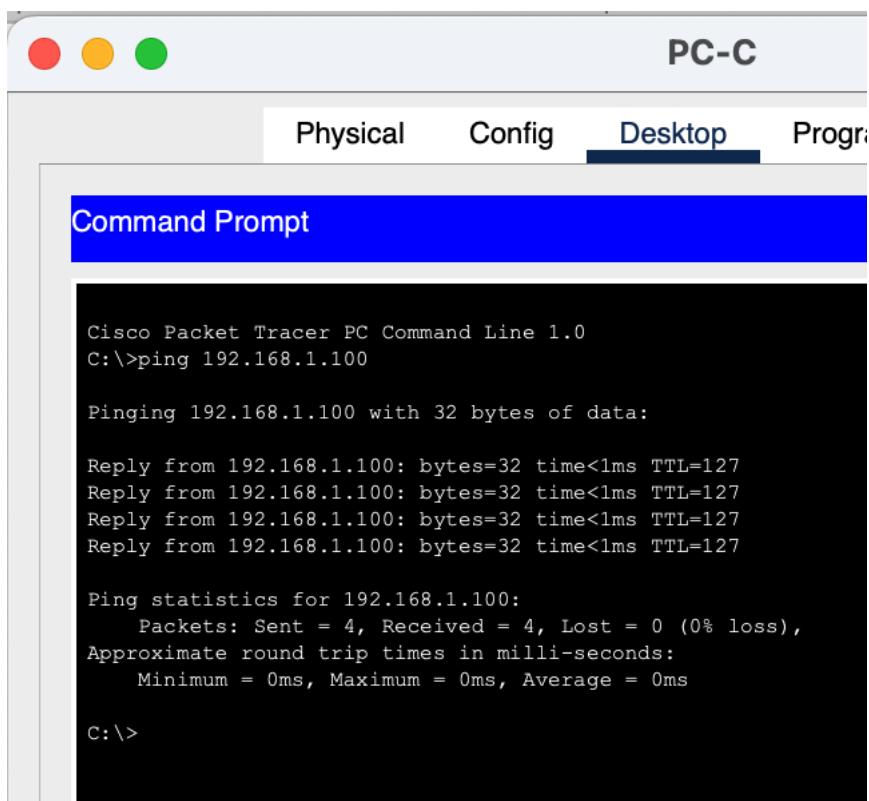


Figure 40: Successful Ping from PC-C/Green LAN

7. Extended ACL: Only Pink to FTP

The final task required an extended ACL to allow *only* Pink LAN to access the FTP server (172.16.1.20) and permit all other traffic.

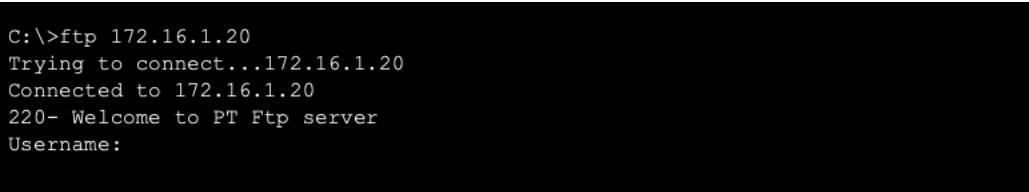
This required a three-line ACL: (1) permit tcp... eq 21 for Pink, (2) deny tcp... eq 21 for all others, and (3) permit ip any any to allow other traffic. I applied this access-list 101 on R2, outbound on the Blue LAN interface (Gig0/1).

Tests confirmed PC-E (Pink) could connect to FTP, but PC-C (Green) was timed out. However, PC-C was still able to ping the server, proving the permit ip any any rule was working.

Screenshots:

```
Router>
Router>enable
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#access-list 101 permit tcp 172.16.2.0 0.0.0.255 host 172.16.1.20 eq 21
Router(config)#access-list 101 deny tcp any host 172.16.1.20 eq 21
Router(config)#access-list 101 permit ip any any
Router(config)#int g0/1
Router(config-if)#ip access-group 101 out
Router(config-if)#end
Router#
%SYS-5-CONFIG_I: Configured from console by console
copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
Router#
```

Figure 41: CLI Configuration of ACL 101 on R2



```
C:\>ftp 172.16.1.20
Trying to connect...172.16.1.20
Connected to 172.16.1.20
220- Welcome to PT Ftp server
Username:
```

Figure 42: Successful FTP from PC-E/ Pink LAN

```

C:\>ftp 172.16.1.20
Trying to connect...172.16.1.20

%Error opening ftp://172.16.1.20/ (Timed out)
.

(Disconnecting from ftp server)

C:\>ping 172.16.1.20

Pinging 172.16.1.20 with 32 bytes of data:

Reply from 172.16.1.20: bytes=32 time=14ms TTL=126
Reply from 172.16.1.20: bytes=32 time=12ms TTL=126
Reply from 172.16.1.20: bytes=32 time<1ms TTL=126
Reply from 172.16.1.20: bytes=32 time<1ms TTL=126

Ping statistics for 172.16.1.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 14ms, Average = 6ms

C:\>|

```

Figure 43: Failed FTP from PC-C/ Green LAN + Successful Ping from PC-C

8. Task B Reflection

This task focused on showing the practical differences between standard and extended ACLs. The main takeaway was understanding the placement rules: standard ACLs which only filter by source are placed close to the destination, while extended ACLs are placed close to the source. This lab also highlighted the importance of rule order and the implicit deny any. The final extended ACL test was a great example of this, requiring a permit, a deny, and a final permit ip any any to create a very specific security policy without breaking other network traffic.

Assessment 1 – Task Sheet C: Site-to-Site IPsec VPN

1. Deliverable 1: VPN Configuration and Reflection

This section outlines the logical flow for configuring a site-to-site IPsec VPN and reflects on the function of each critical component.

1.1) Configuration Flowchart

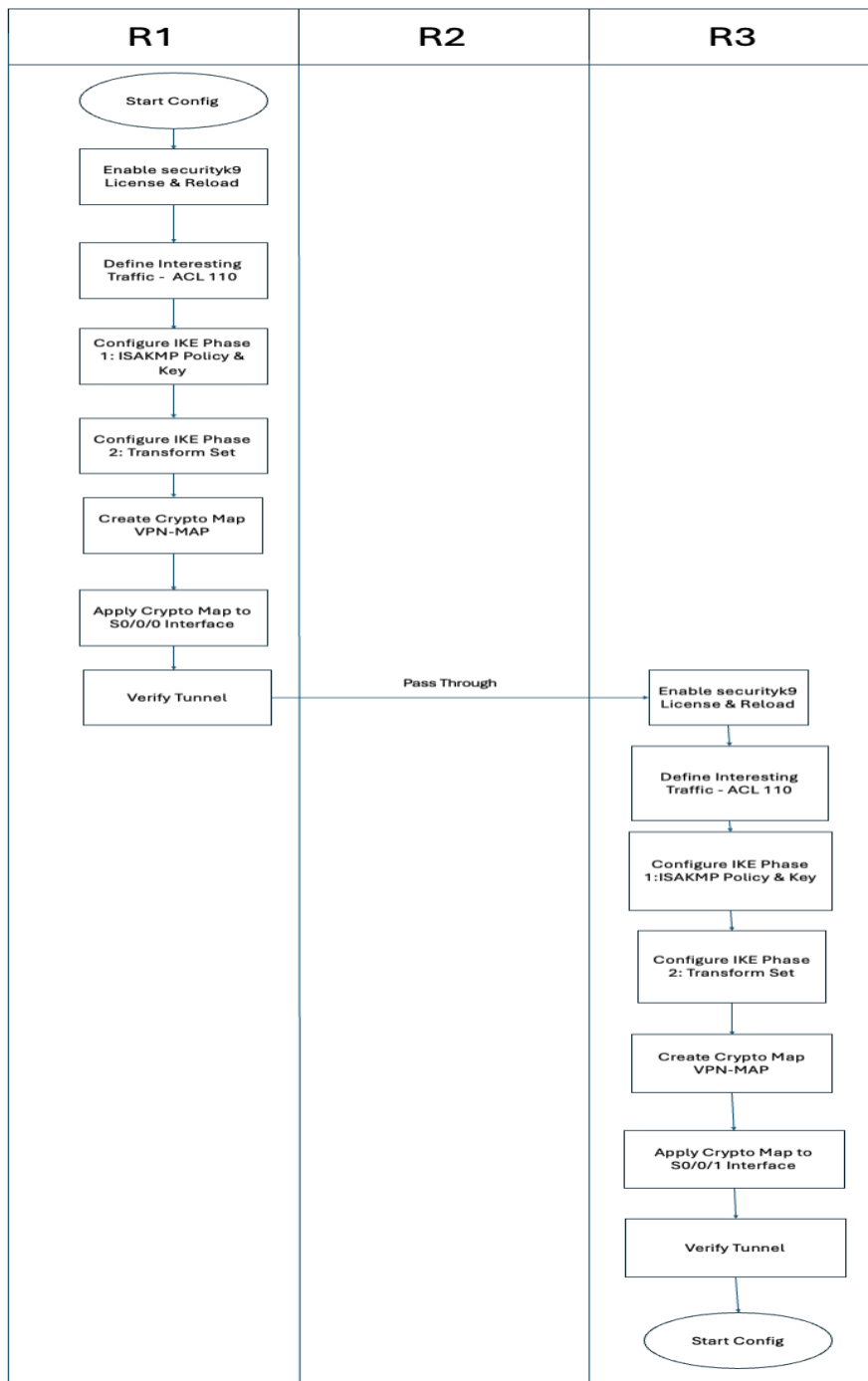


Figure 44: VPN Configuration Flowchart

1.2) Component Functionality Reflection

- Foundation (License & Routing): Enabling the securityk9 license is first step. Its function is to "unlock" the router's advanced security features. Without this license, the IOS would not recognize or process any crypto commands. The underlying OSPF routing is also required, as the VPN *overlay* tunnel needs this *underlay* network to route the encrypted packets between R1 and R3.

- IKE Phase 1 (ISAKMP): This is essentially the management tunnel. Its job is to securely negotiate the keys. The crypto isakmp policy defines rules such as: AES, pre-share, DH group, and the crypto isakmp key defines the password used to authenticate the peer router.
- IKE Phase 2 (IPsec): This is the data tunnel.
 - ACL 110: This is the most important component. Its function is not to block, but to identify the interesting traffic being PC-A LAN to PC-C LAN that should be encrypted.
 - Transform Set: This defines *how* to encrypt the data like esp-aes.
 - Crypto Map: This holds all the components together. It binds the peer's IP, the transform set, and the interesting traffic ACL into a single policy.
- Activation: The crypto map VPN-MAP command is applied to the router's outgoing public interface S0/0/0 on R1 to enable the VPN and start monitoring traffic.

2. Deliverable 2: VPN Functionality Testing

This section describes the testing process used to verify the VPN tunnel's functionality.

Step 1: Baseline test

Before sending any interesting traffic, I issued the show crypto ipsec sa command on R1 to get a baseline. As shown in the screenshot, the packet counters #pkts encaps were at 0, confirming the tunnel was not yet active.

```

^
% Invalid input detected at '^' marker.

R1(config)#R1(config-isakmp)#exit
^
% Invalid input detected at '^' marker.

R1(config)#access-list 110 permit ip 192.168.1.0 0.0.0.255 192.168.3.0 0.0.0.255
R1(config)#crypto isakmp policy 10
R1(config-isakmp)#encryption aes 256
R1(config-isakmp)#authentication pre-share
R1(config-isakmp)#group 5
R1(config-isakmp)#
R1(config-isakmp)#exit
R1(config)#crypto isakmp key vpnpa55 address 10.2.2.2
R1(config)#crypto ipsec transform-set VPN-SET esp-aes esp-sha-hmac
R1(config)#crypto map VPN-MAP 10 ipsec-isakmp
% NOTE: This new crypto map will remain disabled until a peer
and a valid access list have been configured.
R1(config-crypto-map)#description VPN connection to R3
R1(config-crypto-map)#set peer 10.2.2.2
R1(config-crypto-map)#set transform-set VPN-SET
R1(config-crypto-map)#match address 110
R1(config-crypto-map)#exit
R1(config)#interface s0/0/0
R1(config-if)#crypto map VPN-MAP
*Jan  3 07:16:26.785: %CRYPTO-6-ISAKMP_ON_OFF: ISAKMP is ON
R1(config-if)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial0/0/0, changed state to down

00:12:35: %OSPF-5-ADJCHG: Process 101, Nbr 192.168.2.1 on Serial0/0/0 from FULL to DOWN, Neighbor Down: Interface down or detached

%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial0/0/0, changed state to up

00:12:45: %OSPF-5-ADJCHG: Process 101, Nbr 192.168.2.1 on Serial0/0/0 from LOADING to FULL, Loading Done

R1#show crypto ipsec sa

interface: Serial0/0/0
  Crypto map tag: VPN-MAP, local addr 10.1.1.2

  protected vrf: (none)
  local ident (addr/mask/prot/port): (192.168.1.0/255.255.255.0/0/0)
  remote ident (addr/mask/prot/port): (192.168.3.0/255.255.255.0/0/0)
  current_peer 10.2.2.2 port 500
    PERMIT, flags=(origin_is_acl,)
    #pkts encaps: 0, #pkts encrypt: 0, #pkts digest: 0
    #pkts decaps: 0, #pkts decrypt: 0, #pkts verify: 0
    #pkts compressed: 0, #pkts decompressed: 0
    #pkts not compressed: 0, #pkts compr. failed: 0
    #pkts not decompressed: 0, #pkts decompress failed: 0
    #send errors 0, #recv errors 0

    local crypto endpt.: 10.1.1.2, remote crypto endpt.:10.2.2.2
    path mtu 1500, ip mtu 1500, ip mtu idb Serial0/0/0
    current outbound spi: 0x0(0)

  inbound esp sas:

```

Figure 45: R1 CLI Config of baseline counters at 0

Step 2: Testing with Interesting Traffic

I generated interesting traffic by sending a ping from PC-A 192.168.1.3 to PC-C 192.168.3.3, which matches ACL 110. The ping was successful. When I ran show crypto ipsec sa again, the counters were now greater than 0. This proves the router correctly identified the pings, built the IPsec tunnel, and successfully encrypted the data.

```

R1#show crypto ipsec sa

interface: Serial0/0/0
  Crypto map tag: VPN-MAP, local addr 10.1.1.2

  protected vrf: (none)
  local  ident (addr/mask/prot/port): (192.168.1.0/255.255.255.0/0/0)
  remote ident (addr/mask/prot/port): (192.168.3.0/255.255.255.0/0/0)
  current_peer 10.2.2.2 port 500
    PERMIT, flags={origin_is_acl,}
    #pkts encaps: 3, #pkts encrypt: 3, #pkts digest: 0
    #pkts decaps: 2, #pkts decrypt: 2, #pkts verify: 0
    #pkts compressed: 0, #pkts decompressed: 0
    #pkts not compressed: 0, #pkts compr. failed: 0
    #pkts not decompressed: 0, #pkts decompress failed: 0
    #send errors 1, #recv errors 0

  local crypto endpt.: 10.1.1.2, remote crypto endpt.:10.2.2.2
  path mtu 1500, ip mtu 1500, ip mtu idb Serial0/0/0
  current outbound spi: 0x746BD341(1953223489)

  inbound esp sas:
    spi: 0x80A9376D(2158573421)

```

Figure 46: R1 Encaps and decaps >0 interesting traffic

Step 3: Testing with Uninteresting Traffic

Finally, I sent uninteresting traffic by pinging from PC-A 192.168.1.3 to PC-B 192.168.2.3. This ping does not match ACL 110. The ping was successful through normal routing, and when I ran show crypto ipsec sa a final time, the packet counters had not increased. They remained at the same value as in Step 2. This proves the router correctly ignored this traffic and sent it as a normal, unencrypted packet.

```

R1#show crypto ipsec sa

interface: Serial0/0/0
  Crypto map tag: VPN-MAP, local addr 10.1.1.2

  protected vrf: (none)
  local  ident (addr/mask/prot/port): (192.168.1.0/255.255.255.0/0/0)
  remote ident (addr/mask/prot/port): (192.168.3.0/255.255.255.0/0/0)
  current_peer 10.2.2.2 port 500
    PERMIT, flags={origin_is_acl,}
    #pkts encaps: 3, #pkts encrypt: 3, #pkts digest: 0
    #pkts decaps: 2, #pkts decrypt: 2, #pkts verify: 0
    #pkts compressed: 0, #pkts decompressed: 0
    #pkts not compressed: 0, #pkts compr. failed: 0
    #pkts not decompressed: 0, #pkts decompress failed: 0
    #send errors 1, #recv errors 0

  local crypto endpt.: 10.1.1.2, remote crypto endpt.:10.2.2.2
  path mtu 1500, ip mtu 1500, ip mtu idb Serial0/0/0
  current outbound spi: 0x746BD341(1953223489)

  inbound esp sas:
    spi: 0x80A9376D(2158573421)

```

Figure 47: Uninteresting traffic Counter Unchanged

3. Task C Reflection

This task showed the complete way to build an IPsec VPN. The main focus is the two-phase architecture: IKE Phase 1 as a secure management tunnel, and IKE Phase 2 as the data tunnel. The most important concept was the interesting traffic ACL, which functions as a trigger for encryption instead of a filter for blocking. The testing process was also important, as running `show crypto ipsec sa` before and after sending interesting vs. uninteresting traffic was the way to prove the VPN was working exactly as intended.